

Learning To Route

Asaf Valadarsky

Hebrew University of Jerusalem
asaf.valadarsky@mail.huji.ac.il

Dafna Shahaf

Hebrew University of Jerusalem
dafna.shahaf@mail.huji.ac.il

Michael Schapira

Hebrew University of Jerusalem
schapiram@huji.ac.il

Aviv Tamar

UC Berkeley
avivt@berkeley.edu

ABSTRACT

Recently, much attention has been devoted to the question of whether/when traditional network protocol design, which relies on the application of algorithmic insights by human experts, can be replaced by a data-driven (i.e., machine learning) approach. We explore this question in the context of the arguably most fundamental networking task: routing. Can ideas and techniques from machine learning (ML) be leveraged to automatically generate “good” routing configurations? We focus on the classical setting of intradomain traffic engineering. We observe that this context poses significant challenges for data-driven protocol design. Our preliminary results regarding the power of data-driven routing suggest that applying ML (specifically, deep reinforcement learning) to this context yields high performance and is a promising direction for further research. We outline a research agenda for ML-guided routing.

1 INTRODUCTION

Applying machine learning (ML) to computational challenges is prevalent in numerous areas in computer science (AI, computer vision, graphics, NLP, comp-bio, and beyond). Computer networking, in contrast, has largely withstood the ML tide until recently. Recent advances suggest that this might be changing [23, 32, 33].

We ask whether data-driven protocol design [23] can improve upon today’s approaches in the context of routing, i.e., the selection of paths for traffic within a network, or across networks.

Why apply machine learning to routing? Routing is, arguably, the most fundamental networking task and, consequently, has been extensively researched in a broad variety of contexts (data centers, WANs, ISP networks, interdomain

routing with BGP, wireless networks, and more). Traditionally, route-optimization contends with uncertainty about future traffic conditions in one of two manners: (1) optimizing routing configurations with respect to *previously observed* traffic conditions, with the hope that these configurations fare well also with respect to the future, or (2) optimizing with respect to *a range* of feasible traffic scenarios, in hope of providing high performance across the entire range [7, 10, 16, 29].

Unfortunately, in general, routing configurations optimized with respect to specific traffic conditions can fail miserably in achieving good performance even under not-too-different traffic conditions. In addition, optimizing worst-case performance across a broad range of considered traffic scenarios might come at the expense of being far from the achievable optimum for the actual traffic conditions.

Intuitively, ML suggests a third option: leveraging information about *past* traffic conditions to learn good routing configurations for *future* conditions. While the exact future traffic demands are unknown to the decision maker in advance, a realistic assumption is that the history of traffic demands contains *some* information regarding the future (e.g., changes in traffic across times of day, the skewness of traffic, whether certain end-hosts communicate often, etc.). Hence, a natural approach is to continuously observe traffic demands and adapt routing with respect to (implicit or explicit) *predictions* about the future.

Intradomain traffic engineering (TE) as a case study. We initiate the study of ML-guided routing by examining the classical environment of intradomain TE [10, 15–17, 24, 29, 50]—the optimization of routing within a single, self-administered network. We leave the investigation of data-driven routing in other contexts to future research (Section 7).

We present a model for data-driven (intradomain) routing that builds on the rich body of literature on intradomain TE [10, 15–17, 24, 29, 50] and (multicommodity [5, 7, 10, 16, 22, 29, 43]) flow optimization. We investigate, within this model, the application of different ML paradigms and machinery.

In our investigation of ML-guided intradomain TE we grapple with two main questions: (1) How should routing be formulated as an ML problem? and (2) What are suitable representations for the inputs and outputs of learning in this domain? We next expand on each of these challenges, which also pertain to data-driven routing in other contexts.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

HotNets-XVI, November 30–December 1, 2017, Palo Alto, CA, USA

© 2017 Association for Computing Machinery.

ACM ISBN 978-1-4503-5569-8/17/11...\$15.00

<https://doi.org/10.1145/3152434.3152441>

Learn future traffic demands or learn routing configurations? Supervised learning or reinforcement learning? A natural approach to ML-based routing is the following: observe past traffic demands, apply ML to *explicitly* predict the upcoming traffic demands, and optimize routing with respect to the predicted demands. In ML terms, this is a *supervised learning* task [39].

We evaluate several supervised learning schemes for predicting traffic demands. Our preliminary results are discouraging, indicating that supervised learning might be ineffective if the traffic conditions do not exhibit very high regularity. We next turn our attention to a different approach: *reinforcement learning* [45]. Now, instead of explicitly learning future traffic demands and optimizing with respect to these, the goal is to learn a good mapping from the observed history of traffic demands to routing configurations. Our preliminary results suggest that this approach is more promising, yet realizing it requires care, as discussed next.

What should the output of the learning scheme be? The intradomain routing context poses significant challenges to the application of reinforcement learning. A key challenge is that the natural “output” of a routing scheme is a collection of rules specifying how traffic is forwarded from each source to each destination. This output’s naive representation involves a very large set of parameters (as opposed to, e.g., selecting a single action from a fairly small set [32, 33]). Our initial results indicate that this can render learning slow and ineffective. We hence devise methods for constraining the size of the output without losing “too much” in terms of routing expressiveness. We leverage ideas from the literature on hop-by-hop traffic engineering [16, 35, 50] to efficiently learn, via deep reinforcement learning, good routing configurations. Our preliminary findings suggest that this is a promising direction for improving upon today’s intradomain TE.

Outlining a research agenda for data-driven routing. We believe that our investigation below but scratched the surface of data-driven routing. We leave the reader with many interesting research questions, including (1) extending our approach to other routing contexts, (2) examining other performance metrics, (3) identifying better supervised learning approaches to traffic-demand estimation, (4) scaling reinforcement learning in this context, and beyond. We discuss this research agenda in Section 7.

2 DATA-DRIVEN ROUTING MODEL

In our framework, a decision maker (network operator / automated system) repeatedly selects routing configurations. Traffic conditions vary and routing decisions are oblivious to future traffic demands. Our focus is on the conventional optimization objective of minimizing link over-utilization (a.k.a. minimizing congestion) from traffic engineering literature [7, 10, 16, 29].

Network. We model the network as a capacitated directed graph $G = (V, E, c)$, where V and E are the vertex and edge sets, respectively, and $c : E \rightarrow \mathbb{R}^+$ assigns a capacity to each

edge. Let n denote the number of vertices in V and $\Gamma(v)$ denote vertex v ’s neighboring vertices in G .

Routing. A *routing strategy* $\mathcal{R}$ for the network specifies, for each source vertex s and destination vertex t how traffic from s to t that traverses v is split between v ’s neighbors. Thus, a routing strategy specifies, for each vertex v and source-destination pair (s, t) a mapping from v ’s neighbors to values in the interval $[0, 1]$, $\mathcal{R}_{v,(s,t)} : \Gamma(v) \rightarrow [0, 1]$, such that $\mathcal{R}_{v,(s,t)}(u)$ is the fraction of traffic from s to t traversing v that v forwards to its neighbor u . We require that for every $s, t \in V$ and $v \neq t$, $\sum_{u \in \Gamma(v)} \mathcal{R}_{v,(s,t)}(u) = 1$ (no traffic is blackholed at a non-destination), and also for every $s, t \in V$, $\sum_{u \in \Gamma(v)} \mathcal{R}_{t,(s,t)}(u) = 0$ (all traffic to a destination is absorbed at that destination).

Induced flows of traffic. A *demand matrix* (DM) D is a $n \times n$ matrix whose (i, j) ’th entry $D_{i,j}$ specifies the traffic demand between source i and destination j . Observe that any demand matrix D and routing strategy $\mathcal{R}$ induce a flow of traffic in the network, as explained next. Traffic from every source s to destination t is split amongst s ’s neighbors according to $\mathcal{R}_{s,(s,t)}$. Similarly, traffic from s to t traversing a neighbor of s , v , is split amongst v ’s neighbors according to $\mathcal{R}_{v,(s,t)}$, etc.

How good is a traffic flow? We adopt the classical objective function of minimizing link (over)utilization [7, 10, 16, 29]. The link utilization under a specific multicommodity flow f is $\max_{e \in E} \frac{f_e}{c(e)}$, where f_e is the total amount of flow traversing edge e under flow f . Our formulation can easily be extended to other multicommodity-flow-based objective functions. We leave the evaluation of other objectives (e.g., flow-completion time, latency) to future research (Section 7).

We point out that for any *given* demand matrix, computing a multicommodity flow f that minimizes link utilization can be executed in a computationally-efficient manner via linear programming [7, 16, 22]. Our focus, in contrast, is on the realistic scenario in which the DM is not known beforehand.

Routing future traffic demands. Time is divided into consecutive intervals, called “epochs”, of length δ_t (δ_t is determined by the network operator). At the beginning of each epoch t , the routing strategy $\mathcal{R}^{(t)}$ for that epoch is decided. $\mathcal{R}^{(t)}$ can depend only on the history of *past* traffic patterns and routing strategies (from epochs $1, \dots, t-1$).

We make two simplifying assumptions: (1) the demand matrix is fixed throughout each time epoch, and (2) demand matrices can be inferred after the fact (e.g., via network measurements). We leave the investigation of data-driven routing under more complex traffic patterns (e.g., IP flows enter and leave within each epoch) and of information-constrained routing decisions (e.g., only partial information about past traffic demands) to the future.

After selecting the routing strategy $\mathcal{R}^{(t)}$ for epoch t , the demand matrix for epoch t , and the associated cost, in terms of maximum link-utilization, are revealed. The objective of the decision maker is to select routing strategies in a manner that consistently results in low link over-utilization.

3 WHAT TO LEARN?

Our underlying assumption is the existence of some *regularity* in the DMs, and the purpose of the investigation below is exploring how such regularity can be inferred and leveraged to optimize routing. We consider two different manifestations of regularity—embedding *deterministic* regularity into the DM sequence and drawing DMs from a *fixed probability* distribution—and two high-level learning approaches—supervised learning and reinforcement learning.

3.1 Supervised Learning Approach

Since for a given demand matrix (DM), an optimal routing strategy is efficiently computable, a natural approach is to repeatedly try to predict (i.e., learn) the next DM and then compute an optimal routing strategy for that DM. In ML terms, this is a supervised learning problem.

Supervised learning. A supervised learning task involves a sample space $\mathcal{X}$ and a labeling space $\mathcal{Y}$. An algorithm $\mathcal{A}$ for the task is a function mapping values in $\mathcal{X}$ to labels in $\mathcal{Y}$. Given a set of samples and their true labels $\{(x_i, y_i)\} \in \mathcal{X} \times \mathcal{Y}$, the goal is to identify a mapping that produces correct labels for new samples, drawn from the same distribution as the data. How good/bad a mapping fares is quantified in terms of a loss function $\mathcal{L} : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}$. Intuitively, for any pair of labels (y_1, y_2) , $\mathcal{L}(y_1, y_2)$ represents the cost of predicting, for a given sample, the label y_2 instead of the correct label y_1 . See [39] for a detailed exposition of supervised learning.

We consider the following supervised learning approach to routing: the learning algorithm observes the history of DMs up to the current epoch, and predicts the DM for the upcoming epoch. This prediction is then used to generate an optimal routing strategy with respect to the predicted DM. When is employing this scheme a good idea? To answer this question, we evaluate several supervised learning schemes for predicting the next DM on different traffic patterns.

Generating DM sequences for our experiments. We next discuss how traffic patterns are generated in our experiments. We consider two standard schemes for generating DMs: the (deterministic) gravity model [40] and the (probabilistic) bimodal model [34]. Intuitively, the former captures scenarios in which communication between end-points is proportional to their outgoing bandwidths and the latter captures scenarios in which communication end-points are divided into small flows (mice) and large flows (elephants). We also consider “*sparsifications*” of gravity/bimodal DMs generated by selecting, uniformly at random, a p -fraction of the communicating pairs, for some choice of $p \in [0, 1]$, and removing the traffic demands of all other pairs from consideration. We refer to p as the *sparsity* of the DM.

Our experiments require generating *sequences* of DMs, specifying a DM for each time epoch. We examine two classes of DM sequences:

Class I: DM sequences in which the next DM is *deterministically* derived from past DMs. One example for such a DM sequence is “a cycle of DMs”, in which the DM in each

epoch belongs to a fixed set of q DMs, $D^{(0)}, \dots, D^{(q-1)}$ such that if $D^{(j)}$ is the DM in epoch $t - 1$ then $D^{(j+1 \text{ modulo } q)}$ is the DM in epoch t . $D^{(0)}, \dots, D^{(q-1)}$ in our experiments are sparsified (for varied values of p) gravity/bimodal DMs (for varied values for parameters of the bimodal model). Cycles of DMs might capture, e.g., the scenario that the traffic demands at a certain time of day are rather similar across days. See discussion of such temporal consistencies in ISP networks in [17]. Another example of a DM sequence that, though more artificial, also exhibits high regularity (and so is interesting to study) is when each DM is the average of the previous q DMs (for some fixed $q > 0$). Our experiments evaluate supervised learning schemes on DM cycles of sizes $q = 5, 10, 15, 20$, and DM sequences in which each DM is the average over the previous $q = 5, 10, 15, 20$ DMs.

Class II: DM sequences in which each DM is independent of the previous DMs. The DM for each epoch is now drawn independently from a fixed probability distribution over DMs, namely sparsified gravity/bimodal DMs. We point out that such traffic patterns are commonly used in evaluations of data center architectures and protocols [4, 19, 25, 51] as traffic in data centers is often viewed as highly skewed and unpredictable [18, 20].

Supervised learning schemes. Following the recent successes of deep neural networks (DNNs) [28, 38, 41]), we evaluate 3 different DNN architectures. The input to all three architectures is the k most recent observed DMs and the output is a DM. We examine different values of k (5, 10, and 20). We use the Frobenius (or l_2) norm [21] to quantify the quality of an output with respect to the actual DM. The three architectures differ in the structure of the neural network interconnecting the input layer (representing k -long histories of DMs) and output layer (representing the next DM). We evaluate (1) *FCN*, a 3-layered fully-connected network, (2) *CNN*, a 4-layered convolutional neural-network [30], and (3) *NAR-NN*, a nonlinear auto-regressive model [11], realized via a 4-layered neural network that, for input demand matrices $D^{(1)}, \dots, D^{(k)}$, learns a k -vector $\alpha = (\alpha_1, \dots, \alpha_k)$ and an $n \times n$ matrix β , and outputs the DM $\sum_i \alpha_i D^{(i)} + \beta$.

Evaluation framework. We experiment with gravity and bimodal DMs of various sizes (9×9 , 12×12 , 23×23 , 30×30 , 50×50 , and 100×100) under various choices of sparsity levels $p = 0.3, 0.6, 0.9, 1$ and of values of per vertex outgoing bandwidths (ranging from 10’s of MB to 10’s of GB). We consider various DM sequence lengths for training and testing the model (ranging from a few 10’s to few 100’s of DMs). We generate, for each choice of parameter assignment to q, p, k and sequence length, a training set of 10 DM sequences and a test set of 3 DM sequences. We define a *learning epoch* as a full traversal of the training set. We train each neural network for 2000 learning epochs.

Results. Our experimental results (for the test DM sequences) show that for DM sequences that exhibit deterministic regularity, namely, cycles of DMs and “averaged DMs”, only the *NAR-NN* performs fairly well and only for specific relations

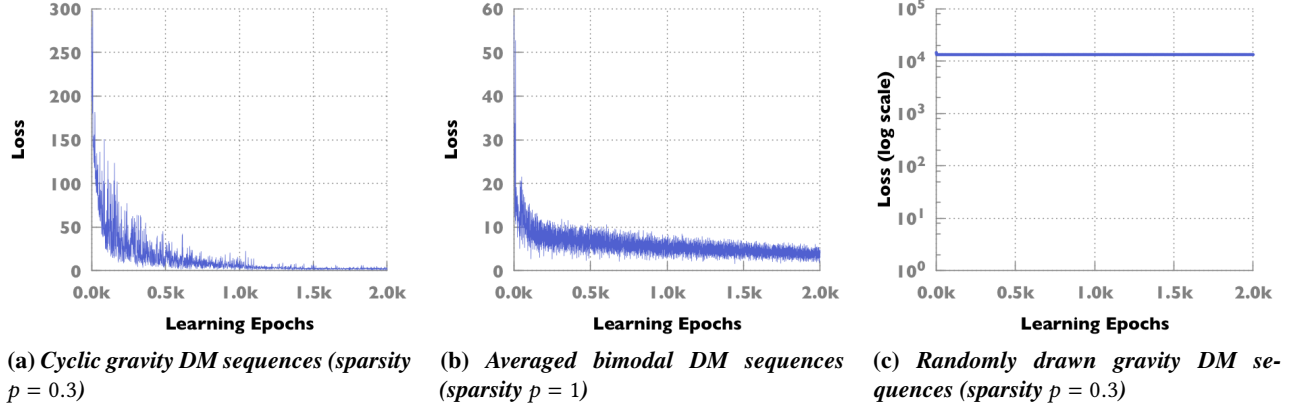


Figure 1: Representative Results for Supervised Learning (using NAR-NN with $k = 10$ and $q = 5$)

between the examined history (k) and the size of the cycle / number of DMs averaged over (q). Specifically, when $q \leq k$, NAR-NN well-approximates the next DM for cycles of DMs, and performs well on averaged DMs. NAR-NN continues to perform reasonably well on averaged DMs when $q > k$, but fails on cycles of DMs for $q > k$. All 3 architectures failed to approximate the next DM for randomly generated DMs (which is not surprising, as there are no temporal correlations between DMs in the sequence).

We present representative results for NAR-NN on a network G with 30 vertices. We plot the loss, in terms of distance of the predicted DM from the actual DM (y-axis), over the number of learning epochs (x-axis). Figure 1b and Figure 1a show that the model succeeds in learning the next DM when using the averaged and cyclic DM sequence generation. Figure 1c demonstrates failure in learning the next DM when drawn from a probability distribution. We leave the investigation of whether better supervised learning of traffic demands is feasible for the future (see Section 7).

3.2 Reinforcement Learning Approach

Next, instead of learning the next DM and optimizing the routing strategy with respect to that DM, our goal is to directly learn a good mapping from observed DMs to routing strategies.

Reinforcement learning. In the reinforcement learning framework, an agent repeatedly interacts with an environment. Time is divided into discrete time slots $t = 1, 2, 3, \dots$. At the beginning of each time slot t , the agent observes the current state s_{t-1} of the environment and selects an action a_t from a fixed set of actions. Once the agent chooses action a_t , the state of the environment changes to s_t and the agent receives a reward r_t (a numerical value) signifying how good/bad the action he took was. The goal of the agent is to learn a mapping π from the set of possible states $\mathcal{S}$ to the space of actions $\mathcal{A}$ (i.e., $\pi : \mathcal{S} \rightarrow \mathcal{A}$) that fares well with respect to the objective of maximizing the *expected discounted reward* $\mathbb{E}[\sum_t \gamma^t r_t]$ for a predetermined $\gamma > 0$, called the *discount factor*. See [45] for a detailed exposition of reinforcement learning.

Routing via reinforcement learning. Routing-strategy selection can be cast as a reinforcement learning task as follows. At the beginning of each time epoch t , the operator/system (agent) decides on a routing strategy $\mathcal{R}^{(t)}$ for that epoch based on the routing strategies and DMs in the most recent k time epochs, which constitute the observed state of the environment at that point. Then, the state changes as the DM for epoch t , $D^{(t)}$, is revealed and the reward $r^{(t)} = -\frac{u^{(t)}}{OPT^{(t)}}$ is received, where $u^{(t)}$ is the max-link-utilization under $\mathcal{R}^{(t)}$ for $D^{(t)}$, and $OPT^{(t)}$ is the optimal max-link-utilization with respect to $D^{(t)}$ ($r^{(t)}$ thus captures the ratio between achieved performance and optimal performance). The goal is to learn a mapping from k -long histories of DMs to routing strategies that maximizes the expected discounted reward, as formulated above. We explore the power of this approach in the following sections.

4 REPRESENTING THE OUTPUT

In contrast to other recent applications of ML to networking [32, 33], learning routing strategies involves generating neural networks with very large output layers (containing, e.g., thousands of output nodes even for a communication network of but tens of vertices). Consider, e.g., the representation of a routing strategy described in Section 2. This representation involves $|V|^2 \cdot |E|$ variables, where $|V|$ and $|E|$ are the sizes of the network graph’s vertex set and edge set, respectively. We show below that even for constrained routing strategies of much smaller sizes, a (“vanilla”) reinforcement learning approach for predicting the complete routing strategy fails to attain good performance within reasonable time.

We restrict our attention to *destination-based* routing strategies, i.e., routing strategies in which the splitting ratios at each vertex u with respect to any destination d are the same across all possible sources s . Observe that any such routing strategy $\mathcal{RS}$ can be represented by $|V| \cdot |E|$ values (i.e., $|V|$ times smaller than unconstrained routing strategies). We employ the continuous-control reinforcement-learning algorithm, TRPO [41], applied to a 3-layered fully-connected neural network, to learn the mapping π from k -long histories

of DMs to a routing strategy $\mathcal{RS}$. The real-valued outputs generated by the deep neural network are turned into per-vertex traffic-splitting ratios by applying, for each vertex in the communication network u , the softmax function [9] to the outputs corresponding to u 's outgoing edges.

Evaluation. We adapt the open-source implementation of TRPO [41] provided by OpenAI [13] to the task of learning routing-strategies. We begin our evaluation with a seemingly easy target: learning the splitting ratios for a 12-vertices, 32-edges network (taken from [26]), and (sparsified) gravity DMs. We train a 3-layered fully-connected network over 7 sequences of gravity DMs of length 60 and evaluate (test) the neural network on 3 such sequences. We repeat this process for sparsity levels 0.3, 0.6, and 0.9. We use $k = 10$ (the length of the history of past DMs received as input). We compute the optimal congestion using the CPLEX [2] LP solver.

The training phase involves generating, from every sequence of DMs of length 60, 50 sequences of 10 consecutive DMs (representing ten-long histories of DMs), by grouping DMs 1 – 10, 2 – 11, etc. Training the neural network on each of these “histories of DMs” involves evaluating the neural network 30 times in parallel (and so 1,500 iterations per DM sequence and 10,500 overall). We refer to one execution of this process as a “learning epoch”.

Our results (omitted due to space constraints) suggest that this approach leads to slow and ineffective learning; e.g., even after more than 700 learning epochs, the produced routing strategies were still over 9x away from the optimum, in terms of max-link-utilization. As shown below, routing strategies that fare significantly better can be generated much quicker. We hypothesize that the large number of output parameters renders efficient learning very challenging. We thus seek a class of routing policies that can be more concisely represented yet is still rich enough to attain high performance.

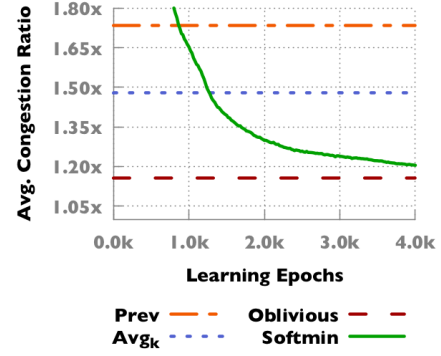
5 LEARNING SOFTMIN ROUTING

We explore the following approach: instead of learning splitting ratios directly, learn *per-edge weights*, and then use these weights to generate a routing strategy. Under this approach, the output of the neural network is of size $|E|$ (as opposed to $|V|^2 \times |E|$ and $|V| \times |E|$ for unrestricted and destination-based routing policies, respectively).

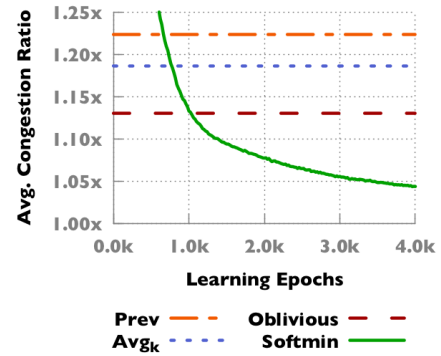
Generating forwarding rules from link weights is a classical approach to routing [16, 35, 50]. We resort to the following approach:

The softmin_γ value for a vector of r coordinates $\alpha = (\alpha_1, \alpha_2, \dots, \alpha_r)$, for $\gamma > 0$, is the vector (also of r coordinates) $\text{softmin}_\gamma(\alpha)_i = \frac{e^{-\gamma \alpha_i}}{\sum_{i=1}^r e^{-\gamma \alpha_i}}$, $i \in 1, \dots, r$. Observe that $\text{softmin}(\alpha)$ can be regarded as a probability distribution (as the sum of all coordinates necessarily equals 1).

Consider a specific assignment of per-edge weights $w = \{w_e\}_{e \in E}$, a specific edge $(u, v) \in E$, and a specific traffic destination d . Observe that, when viewing weights as distances, w determines the length of the shortest path from vertex u to vertex d that goes through u 's immediate neighbor



(a) Congestion ratio for sparse ($p = 0.3$) gravity DM sequences



(b) Congestion ratio for non-sparse ($p = 1.0$) bimodal DM sequences with 40% elephant flows

Figure 2: Representative Results for *softmin*-Routing

v . Let $SP_w(v, u, d)$ denote this length. Given a set of such per-neighbor distances for a vertex u , the *softmin* function can be applied to generate a probability distribution across these neighbors, which can be interpreted as u 's splitting ratios for traffic destined for d . We refer to this scheme as “*softmin-routing*”. The higher the choice of γ to plug into the *softmin* function the closer the resulting routing scheme is to shortest path routing. We set $\gamma = 2$ in our experiments.

Our reinforcement learning scheme maps k -long histories of DMs to per-edge link weights. The reward is computed by turning these weights into traffic splitting ratios and computing the max-link-utilization of the resulting routing strategy with respect to the next DM. We realize this learning scheme via a 3-layers fully-connected network.

We benchmark our results against three alternative non-ML-based approaches to computing routing strategies: (1) *Prev*: optimizing *softmin* routing with respect to the most recent DM, (2) *Avg_k*, optimizing *softmin* routing with respect to the k most recent DMs, and (3) *Oblivious*, the optimal oblivious routing scheme [7] (which does not depend on the history of DMs at all).¹

¹Observe that both *Prev* and *Avg_k* optimize *softmin* routing, as opposed to applying the optimal multicommodity flow computed for the input DM(s)

Evaluation. We consider a communication network with 12 vertices and 32 edges. We use the adaptation of [13] discussed in Section 4 to train a 3-layered fully-connected neural network to generate the weights for softmin routing. We train the neural network on 7 sequences of gravity and bimodal DMs of length 60 each, and tested on 3 such sequences. For gravity DM, the above process is repeated per sparsity levels 0.3, 0.6, and 0.9. For bimodal DMs the sparsity level is $p = 1$ and the percentage of large (elephant) flows amongst the communicating pairs is varied: 20/40/60% of all pairs. We set $k = 10$ and compute the optimal flow via the CPLEX [2] LP solver.

We show in Figure 2a representative results for gravity DMs and in Figure 2b representative results for bimodal based DMs. The figures plot the ratio between the performance of the resulting routing strategies (for the test DM sequences), in terms of averaged max-link-utilization (congestion), and the optimum congestion. Interestingly, oblivious routing outperforms the other two baselines. Observe that *softmin*-routing gets very close to oblivious routing’s performance for gravity DM sequences (and could perhaps outperform it with more training), and significantly outperforms oblivious routing for bimodal DM sequences.

6 RELATED WORK

Traffic engineering Traffic engineering (TE) is fundamental to networking, and hence vastly researched. Results on TE range from routing in legacy, OSPF/ECMP networks [10, 16] to datacenter networks [3] and backbone networks [24]. Softmin routing is inspired by the literature on TE via hop-by-hop routing in IP networks e.g., PEFT [50] and HALO [35]. We find softmin routing especially convenient to use as it involves fairly simple splitting traffic across next-hops, while still achieving high performance.

Reinforcement learning. Machine learning via deep-neural networks has proven extremely useful in executing many different tasks: machine translation [8], image recognition [28], and more. Specifically, reinforcement learning has been applied to playing computer games [37] and beating world-champions in strategic board games [44], robotics [27], 3D-locomotion tasks [41], and beyond. The development and optimization of reinforcement learning algorithms is thus the subject of much attention. Our algorithms rely on utilizing TRPO [41]. We leave the evaluation of other reinforcement-learning algorithms [36, 42, 49] to future research.

ML applications to networking. Machine learning has been applied to various networking contexts including congestion control [12, 48], network bottleneck detection [46], and optimizing datacenter power consumption [1], resource allocation [32], and bitrate selection for video streaming [33].

to route the next DM. The reason is that the latter option is not well defined (and, in particular, some of the end-points communicating in the next DM might not communicate at all in the input DM). We point out that in our experimentation softmin routing is consistently within at most 5% of the optimum traffic flow with respect to any input DM and so very closely approximates this strategy.

Q-routing [31] applies Q-learning [47] to the network routing context. Under Q-routing [31], each router individually learns a mapping from packet headers to outgoing ports. This involves routers constantly exchanging information, at per packet resolution, about their latencies with respect to different destinations. We believe that operating at per packet level, and in a decentralized fashion, poses significant challenges in terms of scalability and communication overhead.

7 CONCLUSION

We initiated the study of data-driven routing and presented preliminary results for the context of intradomain traffic engineering. Our preliminary results from experimentation with deep reinforcement learning show that extracting information from the history of traffic scenarios to generate good routing with respect to future traffic scenarios is an interesting approach. We view our results as a first step towards realizing a much broad research agenda.

Other routing domains. Optimizing routing is a keystone of networking research, investigated in a broad variety of contexts, including legacy IP networks [16], data centers [3], private backbone networks [24], interdomain routing with BGP, overlay networks [6], publish-subscribe networks [14], and more. Applying a data-driven routing approach to other settings is an important research direction.

Other objective functions. Our focus in this study was on the classical objective of minimizing max-link-utilization. Examining other well-studied multicommodity-flow-based objectives, e.g., maximizing overall goodput, is of great interest, as is investigating performance metrics that relate to latency, flow-completion-time, etc.

Predicting traffic-demands. Our preliminary results suggest that well-predicting traffic conditions can, in general, be very challenging. This motivates further research on supervised learning approaches to this challenge.

Better ML-guided intradomain traffic engineering. Our investigation of the application of ML to intradomain TE is only a first step in this direction. Important questions remain regarding (1) the scalability of ML approaches in this context, (2) the environments in which ML-guided routing outperforms traditional routing, and the causes for this, and (3) the “right” choice of the duration of the time epoch be to strike the right balance between routing stability and reactivity to traffic changes.

Better experimental and empirical evaluations. Our experiments involved fairly small networks and synthetically-generated traffic demands. Evaluating routing solutions in more realistic scenarios is important.

ACKNOWLEDGEMENTS

We thank Marco Chiesa for providing us with the code needed to evaluate the oblivious routing scheme. We also thank the anonymous HotNets reviewers for valuable feedback.

REFERENCES

- [1] DeepMind AI Reduces Google Data Centre Cooling Bill by 40%. <https://goo.gl/QTdU2T>.
- [2] IBM ILOG CPLEX Optimizer. <http://www-01.ibm.com/software/commerce/optimization/cplex-optimizer/index.html>.
- [3] M. Al-Fares, S. Radhakrishnan, B. Raghavan, N. Huang, and A. Vahdat. Hedera: Dynamic flow scheduling for data center networks. In *NSDI*, 2010.
- [4] M. Alizadeh, A. Greenberg, D. A. Maltz, J. Padhye, P. Patel, B. Prabhakar, S. Sengupta, and M. Sridharan. Data Center TCP (DCTCP). *ACM SIGCOMM*, 2010.
- [5] M. Allalouf and Y. Shavitt. Maximum flow routing with weighted max-min fairness. In *Quality of Service in the Emerging Networking Panorama*.
- [6] D. Andersen, H. Balakrishnan, F. Kaashoek, and R. Morris. Resilient overlay networks. 2001.
- [7] Y. Azar, E. Cohen, A. Fiat, H. Kaplan, and H. Racke. Optimal oblivious routing in polynomial time. *STOC*, 2003.
- [8] D. Bahdanau, K. Cho, and Y. Bengio. Neural machine translation by jointly learning to align and translate. *arXiv*, 2014.
- [9] C. M. Bishop. *Pattern Recognition and Machine Learning (Information Science and Statistics)*. Springer, 2006.
- [10] M. Chiesa, G. Révész, and M. Schapira. Lying your way to better traffic engineering. *CoNEXT*, 2016.
- [11] T. W. S. Chow and C. T. Leung. Nonlinear autoregressive integrated neural network model for short-term load forecasting. *IEEE Proceedings - Generation, Transmission and Distribution*, 1996.
- [12] M. Dong, Q. Li, D. Zarchy, P. B. Godfrey, and M. Schapira. PCC: Re-architecting congestion control for consistent high performance. *NSDI*, 2015.
- [13] Y. Duan, X. Chen, R. Houthoofd, J. Schulman, and P. Abbeel. Benchmarking deep reinforcement learning for continuous control. *ICML*, 2016.
- [14] P. T. Eugster, P. A. Felber, R. Guerraoui, and A.-M. Kermarrec. The many faces of publish/subscribe. *ACM Comput. Surv.*, 2003.
- [15] B. Fortz, J. Rexford, and M. Thorup. Traffic engineering with traditional ip routing protocols. *Comm. Mag.*, 2002.
- [16] B. Fortz and M. Thorup. Increasing internet capacity using local search. *Computational Optimization and Applications*, 2004.
- [17] B. Fortz and M. Thorup. Optimizing ospf/is-is weights in a changing world. *IEEE J.Sel. A. Commun.*, 2006.
- [18] M. Ghodbadi, R. Mahajan, A. Phanishayee, H. Rastegarfar, P.-A. Blanche, M. Glick, D. Kilper, J. Kulkarni, G. Ranade, and N. Devanur. ProjecToR: Agile Reconfigurable Datacenter Interconnect. *SIGCOMM*, 2016.
- [19] D. Halperin, S. Kandula, J. Padhye, P. Bahl, and D. Wetherall. Augmenting data center networks with multi-gigabit wireless links. *SIGCOMM*, 2011.
- [20] N. Hamedazimi, Z. Qazi, H. Gupta, V. Sekar, S. R. Das, J. P. Longtin, H. Shah, and A. Tanwer. Firefly: A reconfigurable wireless data center fabric using free-space optics. *SIGCOMM*, 2014.
- [21] K. Janocha and W. M. Czarnecki. On loss functions for deep neural networks in classification. *CoRR*, 2017.
- [22] W. S. Jewell. *Multi-commodity Network Solutions*. 1966.
- [23] J. Jiang, V. Sekar, I. Stoica, and H. Zhang. Unleashing the potential of data-driven networking. *COMSNETS*, 2017.
- [24] S. Kandula, D. Katabi, B. Davie, and A. Charny. Walking the tightrope: Responsive yet stable traffic engineering. 2005.
- [25] S. Kassing, A. Valadarsky, G. Shahaf, M. Schapira, and A. Singla. Augmenting data center networks with multi-gigabit wireless links. *SIGCOMM*, 2011.
- [26] S. Knight, H. Nguyen, N. Falkner, R. Bowden, and M. Roughan. The internet topology zoo. *IEEE Journal on Selected Areas in Communications*, 2011.
- [27] J. Kober, J. A. Bagnell, and J. Peters. Reinforcement learning in robotics: A survey. *The International Journal of Robotics Research*, 2013.
- [28] A. Krizhevsky, I. Sutskever, and G. E. Hinton. Imagenet classification with deep convolutional neural networks. In *NIPS*, 2012.
- [29] P. Kumar, Y. Yuan, C. Yu, N. Foster, R. D. Kleinberg, and R. Soulé. Kulfli: Robust traffic engineering using semi-oblivious routing. *CoRR*, 2016.
- [30] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 1998.
- [31] M. Majer, C. Bobda, A. Ahmadiania, and J. Teich. Packet routing in dynamically changing networks on chip. In *Proceedings of the 19th IEEE International Parallel and Distributed Processing Symposium (IPDPS'05) - Workshop 3 - Volume 04*, IPDPS '05, pages 154.2–, Washington, DC, USA, 2005. IEEE Computer Society.
- [32] H. Mao, M. Alizadeh, I. Menache, and S. Kandula. Resource management with deep reinforcement learning. *HotNets*, 2016.
- [33] H. Mao, R. Netravali, and M. Alizadeh. Neural adaptive bitrate streaming with pensive. *SIGCOMM*, 2017.
- [34] A. Medina, N. Taft, K. Salamatian, S. Bhattacharyya, and C. Diot. Traffic matrix estimation: Existing techniques and new directions. 2002.
- [35] N. Michael and A. Tang. Halo: Hop-by-hop adaptive link-state optimal routing. *IEEE/ACM Transactions on Networking*, 2015.
- [36] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu. Asynchronous methods for deep reinforcement learning. *ICML*, 2016.
- [37] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. A. Riedmiller. Playing atari with deep reinforcement learning. *CoRR*, 2013.
- [38] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 2015.
- [39] M. Mohri, A. Rostamizadeh, and A. Talwalkar. *Foundations of Machine Learning*. The MIT Press, 2012.
- [40] M. Roughan, A. Greenberg, C. Kalmanek, M. Rumsewicz, J. Yates, and Y. Zhang. Experience in measuring backbone traffic variability: Models, metrics, measurements and meaning. *IMW*, 2002.
- [41] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz. Trust region policy optimization. *ICML*, 2015.
- [42] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal Policy Optimization Algorithms. *ArXiv e-prints*, 2017.
- [43] F. Shahrokhi and D. W. Matula. The maximum concurrent flow problem. *J. ACM*, 1990.
- [44] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, and D. Hassabis. Mastering the game of Go with deep neural networks and tree search. *Nature*, 2016.
- [45] R. S. Sutton and A. G. Barto. *Reinforcement learning: An introduction*. MIT press Cambridge, 1998.
- [46] H. Wang and B. Li. Lube: Mitigating bottlenecks in wide area data analytics. *HotCloud*, 2017.
- [47] C. J. Watkins and P. Dayan. Q-learning. *Machine Learning*, 1992.
- [48] K. Winstein and H. Balakrishnan. Tcp ex machina: Computer-generated congestion control. *SIGCOMM*, 2013.
- [49] Y. Wu, E. Mansimov, S. Liao, R. B. Grosse, and J. Ba. Scalable trust-region method for deep reinforcement learning using kronecker-factored approximation. *ArXiv e-prints*, 2017.
- [50] D. Xu, M. Chiang, and J. Rexford. Link-state routing with hop-by-hop forwarding can achieve optimal traffic engineering. *INFOCOM*, 2008.
- [51] X. Zhou, Z. Zhang, Y. Zhu, Y. Li, S. Kumar, A. Vahdat, B. Y. Zhao, and H. Zheng. Mirror mirror on the ceiling: Flexible wireless links for data centers. *SIGCOMM*, 2012.